System Verilog Transaction-Based Verification of Dual-Port RAM

Design code

// DUT of Dual-Port Synchronous RAM 256*8

```
module dual_port_ram #(parameter DATA_WIDTH=8, parameter ADDR_WIDTH=8, parameter RAM_DEPTH=256) (
        input logic clk,rst,wr_en_a,rd_en_a,wr_en_b,rd_en_b,
input logic [DATA_WIDTH-1:0] data_in_a,data_in_b,
input logic [ADDR_WIDTH-1:0] addr_a,addr_b,
output logic [DATA_WIDTH-1:0] data_out_a,data_out_b);

logic [DATA_WIDTH-1:0] ram[0:RAM_DEPTH-1];

always_ff @(posedge clk)begin
        if (rst) begin
                data_out_a <= '0;
                data_out_b <= '0;
        end
        else begin
                if (rd_en_a)
                   data_out_a <= ram[addr_a];
                if (wr_en_a)
                        ram[addr_a] <= data_in_a;

                if (rd_en_b)
                   data_out_b <= ram[addr_b];
            if (wr_en_b && !(wr_en_a && (addr_a==addr_b)))
```

```
                ram[addr_b] <= data_in_b;


        end
end
endmodule
```

Interface: -

// interface of DUAL-Port RAM

```
interface ram_if #(parameter DATA_WIDTH=8, ADDR_WIDTH=8)(input logic clk);
        logic rst;
        logic wr_en_a,rd_en_a;
        logic wr_en_b,rd_en_b;
        logic [DATA_WIDTH-1:0] data_in_a,data_in_b;
        logic [ADDR_WIDTH-1:0] addr_a,addr_b;
        logic [DATA_WIDTH-1:0] data_out_a,data_out_b;


        modport DUT (input
clk,rst,wr_en_a,rd_en_a,wr_en_b,rd_en_b,data_in_a,data_in_b,addr_a,addr_b,output
data_out_a,data_out_b);


        modport TB (input clk,data_out_a,data_out_b,output
rst,wr_en_a,rd_en_a,wr_en_b,rd_en_b,data_in_a,data_in_b,addr_a,addr_b);


endinterface
```

// Packet /Transaction

```
//
class ram_packet;
```

```
    rand bit wr_en_a;
    rand bit rd_en_a;
    rand bit wr_en_b;
    rand bit rd_en_b;

    rand bit [7:0] data_in_a;
    rand bit [7:0] data_in_b;
    rand bit [7:0] addr_a;
    rand bit [7:0] addr_b;

    bit [7:0] data_out_a;
    bit [7:0] data_out_b;

    function void display(string class_name);
        $display("[class name=%s] wr_en_a=%0b rd_en_a=%0b wr_en_b=%0b rd_en_b=%0b data_in_a=%h data_in_b=%h addr_a=%0d addr_b=%0d",class_name,wr_en_a,rd_en_a,wr_en_b,rd_en_b,data_in_a,data_in_b,addr_a,addr_b);
    endfunction

endclass
```

// Generator of RAM

```
//
class generator;
    mailbox #(ram_packet) gen2drv;

    function new(mailbox #(ram_packet) mbx);
```

```
        this.gen2drv=mbx;
    endfunction

    task run();
        ram_packet pkt;

        int i;
        bit [7:0] addr_q[$];
        bit [7:0] addr;

        $display("[GEN] : WRITE DATA HAS BEEN STARTED IN MEMORY: PORT_A");
        repeat(10) begin
            pkt=new();
            pkt.randomize() with { wr_en_a == 1; rd_en_a == 0 ; wr_en_b == 0; rd_en_b == 0 ;};
            addr_q.push_back(pkt.addr_a);
            gen2drv.put(pkt);
            pkt.display("GEN:WR_PORT_A");
        end

        $display("[GEN] : READ DATA HAS BEEN STARTED FROM MEMORY: PORT_A");
        foreach (addr_q[i]) begin
            pkt = new();
            pkt.wr_en_a    = 0;
            pkt.rd_en_a    = 1;
            pkt.addr_a     = addr_q[i];
```

```
            gen2drv.put(pkt);
            pkt.display("GEN-RD_PORT_A");
        end

        $display("[GEN] : WRITE-READ DATA HAS BEEN STARTED IN
MEMORY: PORT_A");
        repeat (5) begin
            pkt = new();
            pkt.randomize() with {
                wr_en_a  == 1;
                rd_en_a  == 0;
                wr_en_b  == 0;
                rd_en_b  == 0;
            };
            addr = pkt.addr_a;
            gen2drv.put(pkt);
            pkt.display("GEN-WR_PORT_A");

            pkt = new();
            pkt.wr_en_a    = 0;
            pkt.rd_en_a    = 1;
            pkt.addr_a     = addr;
            gen2drv.put(pkt);
            pkt.display("GEN-RD_PORT_A");
        end

        $display("[GEN] : WRITE DATA HAS BEEN STARTED IN MEMORY:
PORT_B");
```

```
            for (i = 0; i < 10; i++) begin
                pkt = new();
                pkt.randomize() with {wr_en_a == 0; rd_en_a  == 0; wr_en_b == 1; rd_en_b == 0; addr_b == i;};
                gen2drv.put(pkt);
                pkt.display("GEN-WR_PORT_B");
            end

        $display("[GEN] : READ DATA HAS BEEN STARTED IN MEMORY: PORT_B");
            for (i = 0; i < 10; i++) begin
                pkt = new();
                pkt.wr_en_b  = 0;
                pkt.rd_en_b  = 1;
                pkt.addr_b   = i;
                gen2drv.put(pkt);
                pkt.display("GEN-RD_PORT_B");
            end

        $display("[GEN] : WRITE DATA HAS BEEN STARTED IN MEMORY: PORT_B");
        for (i = 0; i < 5; i++) begin
            pkt = new();
            pkt.randomize() with {wr_en_a == 0; rd_en_a  == 0; wr_en_b == 1; rd_en_b == 0; addr_b== i; data_in_b == 8'hA0 + 2*i ;};
            gen2drv.put(pkt);
            pkt.display("GEN-WR_PORT_B");
        end
```

```
        $display("[GEN] : READ DATA HAS BEEN STARTED IN MEMORY: PORT_B");
        for (i = 0; i < 5; i++) begin
            pkt = new();
            pkt.wr_en_b  = 0;
            pkt.rd_en_b  = 1;
            pkt.addr_b   = i;
            gen2drv.put(pkt);
            pkt.display("GEN-RD_PORT_B");
        end

        $display("[GEN] : OVER_WRITE ONLY 3 DATA HAS BEEN STARTED IN MEMORY: PORT_B");
        for (i = 0; i < 3; i++) begin
            pkt = new();
            pkt.randomize() with {wr_en_a == 0; rd_en_a  == 0; wr_en_b == 1; rd_en_b == 0; addr_b == i; data_in_b == 8'hB0 + 2*i ;};
            gen2drv.put(pkt);
            pkt.display("GEN-OVER_WR_PORT_B");
        end

        $display("[GEN] : READ the 3 NEW UPDATED DATA HAS BEEN STARTED IN MEMORY: PORT_B");
        for (i = 0; i < 7; i++) begin
            pkt = new();
            pkt.wr_en_b  = 0;
            pkt.rd_en_b  = 1;
            pkt.addr_b   = i;
            gen2drv.put(pkt);
```

```
            pkt.display("GEN-OVER_RD_PORT_B");
        end

        $display("[GEN] : WRITE DATA HAS BEEN STARTED AT DIFF. ADDRESS IN MEMORY:PORT_A & PORT_B");
        for (i = 0; i < 5; i++) begin
            pkt = new();
            pkt.randomize() with {wr_en_a == 1; rd_en_a  == 0; wr_en_b == 1; rd_en_b == 0; addr_a == i; addr_b == 10+i;};
            gen2drv.put(pkt);
            pkt.display("GEN-WR_PORT_A&B");
        end

        $display("[GEN] : READ DATA HAS BEEN STARTED FROM DIFF. ADDRESS IN MEMORY:PORT_A & PORT_B");
        for (i = 0; i < 5; i++) begin
            pkt = new();
                pkt.wr_en_a  = 0;
            pkt.wr_en_b  = 0;
                pkt.rd_en_a  = 1;
            pkt.rd_en_b  = 1;
                pkt.addr_a   = i;
            pkt.addr_b   = 10+i;
            gen2drv.put(pkt);
            pkt.display("GEN-RD_PORT_A&B");
        end

        $display("[GEN] : WRITE DATA HAS BEEN STARTED AT SAME ADDRESS IN MEMORY:PORT_A & PORT_B");
```

```
        for (i = 0; i < 5; i++) begin
            pkt = new();
            pkt.randomize() with {wr_en_a == 1; rd_en_a  == 0; wr_en_b == 1;
rd_en_b == 0; addr_a == i; addr_b == i; data_in_a == 70+i; data_in_b == 80+i;};
            gen2drv.put(pkt);
            pkt.display("GEN-WR_PORT_A&B");
        end

        $display("[GEN] : READ DATA HAS BEEN STARTED FROM SAME
ADDRESS IN MEMORY:PORT_A & PORT_B");
        for (i = 0; i < 5; i++) begin
            pkt = new();
            pkt.wr_en_a  = 0;
            pkt.wr_en_b  = 0;
            pkt.rd_en_a  = 1;
            pkt.rd_en_b  = 0;
            pkt.addr_a   = i;
            pkt.addr_b   = i;
            gen2drv.put(pkt);
            pkt.display("GEN-RD_PORT_A&B");
        end

    endtask
endclass
```

// Driver for RAM

```
//
class driver;
```

```
mailbox #(ram_packet) gen2drv;
virtual ram_if.TB vif;

function new(mailbox #(ram_packet) mbx, virtual ram_if.TB vif);
        this.gen2drv=mbx;
        this.vif=vif;
endfunction

task run();
        ram_packet pkt;
        wait(vif.rst == 0);

        forever begin
                gen2drv.get(pkt);
                @(posedge vif.clk);
                vif.wr_en_a <= pkt.wr_en_a;
                vif.rd_en_a <= pkt.rd_en_a;
                vif.wr_en_b <= pkt.wr_en_b;
                vif.rd_en_b <= pkt.rd_en_b;
                vif.addr_a  <= pkt.addr_a;
                vif.addr_b  <= pkt.addr_b;

                vif.data_in_a <= pkt.data_in_a;
                vif.data_in_b <= pkt.data_in_b;

                if (pkt.wr_en_a) begin
                        $display("[DRV] WRITE DATA_PORT_A : Time=%0t
wr_en_a=%0b DATA=%h addr_a=%0d",$time,pkt.wr_en_a,pkt.data_in_a,pkt.addr_a);
```

```
        end

        @(posedge vif.clk);
        vif.wr_en_a <= 0;
        vif.rd_en_a <= 0;
        vif.wr_en_b <= 0;
        vif.rd_en_b <= 0;

        if (pkt.rd_en_a) begin
            @(posedge vif.clk);
            $display("[DRV] READ DATA_PORT_A OK : Time=%0t Addr=%0d Data=%h",$time, pkt.addr_a, vif.data_out_a);
        end

    if (pkt.wr_en_b) begin
        $display("[DRV] WRITE DATA_PORT_B : Time=%0t wr_en_b=%0b DATA=%h addr_b=%0d",$time,pkt.wr_en_b,pkt.data_in_b,pkt.addr_b);
    end

    @(posedge vif.clk);
    vif.wr_en_a <= 0;
    vif.rd_en_a <= 0;
    vif.wr_en_b <= 0;
    vif.rd_en_b <= 0;

    if (pkt.rd_en_b) begin
        @(posedge vif.clk);
```

```
                    $display("[DRV] READ DATA_PORT_B OK : Time=%0t Addr=%0d Data=%h",$time, pkt.addr_b, vif.data_out_b);
                end

            end

        endtask
endclass
```

//Monitor of RAM

```
//
class monitor;
    virtual ram_if.TB vif;
    mailbox #(ram_packet) mon2scb;
    ram_coverage cov;

    function new(mailbox #(ram_packet) mbx,virtual ram_if.TB vif);
        this.mon2scb=mbx;
        this.vif=vif;
        cov = new();
    endfunction

    task run();
        ram_packet pkt;

        $display("START Printing the Monitor DATA");
        forever begin
```

```
                @(posedge vif.clk);
                if(vif.rst) begin
                        $display("[MON] RESET ACTIVE -
IGNORING_TRANSACTIONS");
                        continue;
                end

                if (vif.wr_en_a || vif.wr_en_b) begin
                        pkt = new();
                        pkt.wr_en_a      = 1;
                        pkt.wr_en_b      = 1;
                        pkt.rd_en_a      = 0;
                        pkt.rd_en_b      = 0;
                        pkt.addr_a       = vif.addr_a;
                        pkt.addr_b       = vif.addr_b;
                        pkt.data_in_a    = vif.data_in_a;
                        pkt.data_in_b    = vif.data_in_b;
                        $display("[MON-WR] PORT_A_WRITE Addr=%0d
Data=%h",pkt.addr_a, pkt.data_in_a);
                        cov.ram_cg.sample(pkt);
                        mon2scb.put(pkt);
                end

                if (vif.rd_en_a) begin
                        @(posedge vif.clk);
                        pkt = new();
                        pkt.wr_en_a      = 0;
                        pkt.rd_en_a      = 1;
```

```
                        pkt.addr_a        = vif.addr_a;
                        pkt.data_out_a    = vif.data_out_a;
                        $display("[MON-RD] PORT_A_READ Addr=%0d Data=%h",
pkt.addr_a, pkt.data_out_a);
                        cov.ram_cg.sample(pkt);
                        mon2scb.put(pkt);
                    end

/*                  if (vif.wr_en_b) begin
                        pkt = new();
                        pkt.wr_en_b       = 1;
                        pkt.rd_en_b       = 0;
                        pkt.addr_b        = vif.addr_b;
                        pkt.data_in_b     = vif.data_in_b;
                        $display("[MON-WR] PORT_B_WRITE Addr=%0d
Data=%h",pkt.addr_b, pkt.data_in_b);
        //                cov.ram_cg.sample(pkt);
                        mon2scb.put(pkt);
                end
*/
                if (vif.rd_en_b) begin
                        @(posedge vif.clk);
                        pkt = new();
                        pkt.wr_en_b       = 0;
                        pkt.rd_en_b       = 1;
                        pkt.addr_b        = vif.addr_b;
                        pkt.data_out_b      = vif.data_out_b;
```

```
                $display("[MON-RD] PORT_B_READ Addr=%0d Data=%h", pkt.addr_b, pkt.data_out_b);
                cov.ram_cg.sample(pkt);
                mon2scb.put(pkt);
            end

        end
    endtask
endclass
```

// Scoreboard of RAM

```
//
class scoreboard;
    mailbox #(ram_packet) mon2scb;
    bit [7:0] ref_mem [0:255];

    function new(mailbox #(ram_packet) mbx);
        this.mon2scb=mbx;
    endfunction

    task run();
        ram_packet pkt;
        forever begin
            mon2scb.get(pkt);

            if(pkt.wr_en_a == 0 && pkt.wr_en_b == 0 && pkt.rd_en_a == 0 && pkt.rd_en_b == 0) begin
                ref_mem = '{default:0};
```

```
                $display("[SCB] RESET DETECTED - REF MEM
CLEARED");
                continue;
            end

            if (pkt.wr_en_a) begin
                ref_mem[pkt.addr_a] = pkt.data_in_a ;
                $display("[SCB] PORT_A STORE DATA Addr=%0d
Data=%h",pkt.addr_a, pkt.data_in_a);
            end

        if (pkt.wr_en_b && !(pkt.wr_en_a && (pkt.addr_a==pkt.addr_b))) begin
            ref_mem[pkt.addr_b] = pkt.data_in_b ;
            $display("[SCB] PORT_B STORE DATA Addr=%0d
Data=%h",pkt.addr_b, pkt.data_in_b);
        end


            if (pkt.rd_en_a) begin
                if (pkt.data_out_a == ref_mem[pkt.addr_a]) begin
                    $display("[SCB] PORT_A [PASS] Addr=%0d
Data=%h", pkt.addr_a, pkt.data_out_a);
                end
                else begin
                    $error("[SCB][FAIL] Addr=%0d Exp=%h Got=%h",
pkt.addr_a, ref_mem[pkt.addr_a], pkt.data_out_a);
                end
            end
```

```
                if (pkt.rd_en_b) begin
                    if (pkt.data_out_b == ref_mem[pkt.addr_b]) begin
                        $display("[SCB] PORT_B [PASS] Addr=%0d Data=%h",
pkt.addr_b, pkt.data_out_b);
                    end
                    else begin
                        $error("[SCB][FAIL] Addr=%0d Exp=%h Got=%h",
pkt.addr_b, ref_mem[pkt.addr_b], pkt.data_out_b);
                    end
                end

            end
        endtask
endclass
```

// Testbench of RAM

```
//
`include "Dual_portram.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "coverage.sv"
`include "monitor.sv"
`include "scoreboard.sv"
`include "assertions.sv"

module tb;
```

```
logic clk=0;

always #5 clk=~clk;

ram_if intf(clk);

dual_port_ram
dut(.clk(clk),.rst(intf.rst),.wr_en_a(intf.wr_en_a),.rd_en_a(intf.rd_en_a),.wr_en_b(intf.wr_en_b),.rd_en_b(intf.rd_en_b),.data_in_a(intf.data_in_a),.data_in_b(intf.data_in_b),.addr_a(intf.addr_a),.addr_b(intf.addr_b),.data_out_a(intf.data_out_a),.data_out_b(intf.data_out_b));

ram_assertions u_ram_assertions
(.clk(clk),.rst(intf.rst),.wr_en_a(intf.wr_en_a),.rd_en_a(intf.rd_en_a),.wr_en_b(intf.wr_en_b),.rd_en_b(intf.rd_en_b),.addr_a(intf.addr_a),.addr_b(intf.addr_b),.data_in_a(intf.data_in_a),.data_in_b(intf.data_in_b),.data_out_a(intf.data_out_a),.data_out_b(intf.data_out_b));

mailbox #(ram_packet) gen2drv;

mailbox #(ram_packet) mon2scb;

generator gen;

driver drv;

monitor mon;

scoreboard scb;

initial begin

        intf.rst = 1;
```

```
intf.wr_en_a = 0;
intf.rd_en_a = 0;
intf.wr_en_b = 0;
intf.rd_en_b = 0;

intf.addr_a = 0;
intf.addr_b = 0;

intf.data_in_a = 0;
intf.data_in_b = 0;

$display("================================");
$display("RESET ASSERTED");
$display("================================");

repeat(5) @(posedge clk);

intf.rst = 0;

$display("================================");
$display("RESET DEASSERTED - START TEST");
$display("================================");

gen2drv=new();
mon2scb=new();

gen=new(gen2drv);
```

```
        drv=new(gen2drv,intf);
        mon=new(mon2scb,intf);
        scb=new(mon2scb);

        fork
                gen.run();
                drv.run();
                mon.run();
                scb.run();
        join_none

        #4000;
        $display("ALL Test has been succesfully completed!!!");
        $finish;
end

initial begin
        $fsdbDumpfile("ram.fsdb");
        $fsdbDumpvars(0,tb);
end

endmodule
```

// Coverage of RAM

```
//
class ram_coverage;
        ram_packet pkt;
```

```
    covergroup ram_cg with function sample(ram_packet pkt);

cp_wr_a : coverpoint pkt.wr_en_a {
   bins write_a = {1};
}

cp_rd_a : coverpoint pkt.rd_en_a {
   bins read_a = {1};
}

cp_wr_b : coverpoint pkt.wr_en_b {
   bins write_b = {1};
}

cp_rd_b : coverpoint pkt.rd_en_b {
   bins read_b = {1};
}

cp_addr_a : coverpoint pkt.addr_a {
   bins low_addr  = {[0:63]};
   bins mid_addr  = {[64:127]};
   bins high_addr = {[128:255]};
}

cp_addr_b : coverpoint pkt.addr_b {
   bins low_addr  = {[0:63]};
```

```
    bins mid_addr  = {[64:127]};
    bins high_addr = {[128:255]};
  }

  cp_data_a : coverpoint pkt.data_in_a {
    bins low_data  = {[0:63]};
    bins mid_data  = {[64:127]};
    bins high_data = {[128:255]};
  }

  cp_data_b : coverpoint pkt.data_in_b {
    bins low_data  = {[0:63]};
    bins mid_data  = {[64:127]};
    bins high_data = {[128:255]};
  }

  cross_wr_ports : cross cp_wr_a, cp_wr_b;
  cross_rd_ports : cross cp_rd_a, cp_rd_b;

  cp_addr_conflict : coverpoint (pkt.addr_a == pkt.addr_b) {
    bins same_addr = {1};
    bins diff_addr = {0};
  }

  cross_wr_conflict : cross cp_wr_a, cp_wr_b, cp_addr_conflict;
endgroup
```

```
function new();
        ram_cg = new();
endfunction
endclass
```

Run command

[b23n_sarif_727@prac3 dualportram]$ vcs -sverilog top.sv -kdb -lca -debug_access+all -full64

./simv

./simv -verdi

verdi -cov -covdir simv.vdb/

File name: -

[b23n_sarif_727@prac3 dualportram]$ ls

assertions.sv Dual_portram.sv novas_dump.log scoreboard.sv top.sv

coverage.sv generator.sv novas.rc simv ucli.key

csrc interface.sv packet.sv simv.daidir vdCov.conf

driver.sv monitor.sv ram.fsdb simv.vdb vdCovLog

[b23n_sarif_727@prac3 dualportram]$ verdi -cov -covdir simv.vdb/

Simulation Result

Output

[GEN] : WRITE-READ DATA HAS BEEN STARTED IN MEMORY: PORT_A

[class name=GEN-WR_PORT_A] wr_en_a=1 rd_en_a=0 wr_en_b=0 rd_en_b=0 data_in_a=4e data_in_b=b7 addr_a=11 addr_b=134

[class name=GEN-RD_PORT_A] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=11 addr_b=0

[class name=GEN-WR_PORT_A] wr_en_a=1 rd_en_a=0 wr_en_b=0 rd_en_b=0 data_in_a=84 data_in_b=30 addr_a=209 addr_b=81

[class name=GEN-RD_PORT_A] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=209 addr_b=0

[class name=GEN-WR_PORT_A] wr_en_a=1 rd_en_a=0 wr_en_b=0 rd_en_b=0 data_in_a=07 data_in_b=85 addr_a=238 addr_b=255

[class name=GEN-RD_PORT_A] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=238 addr_b=0

[class name=GEN-WR_PORT_A] wr_en_a=1 rd_en_a=0 wr_en_b=0 rd_en_b=0 data_in_a=d3 data_in_b=21 addr_a=116 addr_b=43

[class name=GEN-RD_PORT_A] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=116 addr_b=0

[class name=GEN-WR_PORT_A] wr_en_a=1 rd_en_a=0 wr_en_b=0 rd_en_b=0 data_in_a=93 data_in_b=44 addr_a=200 addr_b=48

[class name=GEN-RD_PORT_A] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=200 addr_b=0

[GEN] : WRITE DATA HAS BEEN STARTED IN MEMORY: PORT_B

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=0b data_in_b=64 addr_a=173 addr_b=0

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=5d data_in_b=f9 addr_a=136 addr_b=1

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=fa data_in_b=a8 addr_a=79 addr_b=2

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=4a data_in_b=f1 addr_a=184 addr_b=3

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=ae data_in_b=fc addr_a=202 addr_b=4

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=db data_in_b=26 addr_a=161 addr_b=5

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=7c data_in_b=28 addr_a=26 addr_b=6

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=24 data_in_b=e6 addr_a=178 addr_b=7

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=44 data_in_b=90 addr_a=140 addr_b=8

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=b9 data_in_b=47 addr_a=140 addr_b=9

[GEN] : READ DATA HAS BEEN STARTED IN MEMORY: PORT_B

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=0

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=1

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=2

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=3

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=4

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=5

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=6

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=7

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=8

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=9

[GEN] : WRITE DATA HAS BEEN STARTED IN MEMORY: PORT_B

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=f5 data_in_b=a0 addr_a=53 addr_b=0

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=b4 data_in_b=a2 addr_a=202 addr_b=1

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=e9 data_in_b=a4 addr_a=14 addr_b=2

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=e2 data_in_b=a6 addr_a=211 addr_b=3

[class name=GEN-WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=d5 data_in_b=a8 addr_a=102 addr_b=4

[GEN] : READ DATA HAS BEEN STARTED IN MEMORY: PORT_B

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=0

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=1

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=2

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=3

[class name=GEN-RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=4

[GEN] : OVER_WRITE ONLY 3 DATA HAS BEEN STARTED IN MEMORY: PORT_B

[class name=GEN-OVER_WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=1f data_in_b=b0 addr_a=83 addr_b=0

[class name=GEN-OVER_WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=d4 data_in_b=b2 addr_a=79 addr_b=1

[class name=GEN-OVER_WR_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=9e data_in_b=b4 addr_a=246 addr_b=2

[GEN] : READ the 3 NEW UPDATED DATA HAS BEEN STARTED IN MEMORY: PORT_B

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=0

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=1

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=2

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=3

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=4

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=5

[class name=GEN-OVER_RD_PORT_B] wr_en_a=0 rd_en_a=0 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=6

[GEN] : WRITE DATA HAS BEEN STARTED AT DIFF. ADDRESS IN MEMORY:PORT_A & PORT_B

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=a4 data_in_b=bc addr_a=0 addr_b=10

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=85 data_in_b=7d addr_a=1 addr_b=11

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=a9 data_in_b=93 addr_a=2 addr_b=12

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=07 data_in_b=6b addr_a=3 addr_b=13

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=70 data_in_b=eb addr_a=4 addr_b=14

[GEN] : READ DATA HAS BEEN STARTED FROM DIFF. ADDRESS IN MEMORY:PORT_A & PORT_B

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=0 addr_b=10

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=1 addr_b=11

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=2 addr_b=12

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=3 addr_b=13

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=1 data_in_a=00 data_in_b=00 addr_a=4 addr_b=14

[GEN] : WRITE DATA HAS BEEN STARTED AT SAME ADDRESS IN MEMORY:PORT_A & PORT_B

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=46 data_in_b=50 addr_a=0 addr_b=0

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=47 data_in_b=51 addr_a=1 addr_b=1

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=48 data_in_b=52 addr_a=2 addr_b=2

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=49 data_in_b=53 addr_a=3 addr_b=3

[class name=GEN-WR_PORT_A&B] wr_en_a=1 rd_en_a=0 wr_en_b=1 rd_en_b=0 data_in_a=4a data_in_b=54 addr_a=4 addr_b=4

[GEN] : READ DATA HAS BEEN STARTED FROM SAME ADDRESS IN MEMORY:PORT_A & PORT_B

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=0 addr_b=0

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=1 addr_b=1

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=2 addr_b=2

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=3 addr_b=3

[class name=GEN-RD_PORT_A&B] wr_en_a=0 rd_en_a=1 wr_en_b=0 rd_en_b=0 data_in_a=00 data_in_b=00 addr_a=4 addr_b=4

START Printing the Monitor DATA

[ASSERT PASS] Reset clears outputs

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=55 wr_en_a=1 DATA=98 addr_a=101

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=101 Data=98

[SCB] PORT_A STORE DATA Addr=101 Data=98

[SCB] PORT_B STORE DATA Addr=229 Data=89

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=85 wr_en_a=1 DATA=d5 addr_a=237

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=237 Data=d5

[SCB] PORT_A STORE DATA Addr=237 Data=d5

[SCB] PORT_B STORE DATA Addr=4 Data=e1

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=115 wr_en_a=1 DATA=06 addr_a=14

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=14 Data=06

[SCB] PORT_A STORE DATA Addr=14 Data=06

[SCB] PORT_B STORE DATA Addr=63 Data=e1

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=145 wr_en_a=1 DATA=fb addr_a=226

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=226 Data=fb

[SCB] PORT_A STORE DATA Addr=226 Data=fb

[SCB] PORT_B STORE DATA Addr=86 Data=9a

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=175 wr_en_a=1 DATA=2e addr_a=141

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=141 Data=2e

[SCB] PORT_A STORE DATA Addr=141 Data=2e

[SCB] PORT_B STORE DATA Addr=246 Data=55

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=205 wr_en_a=1 DATA=f5 addr_a=197

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=197 Data=f5

[SCB] PORT_A STORE DATA Addr=197 Data=f5

[SCB] PORT_B STORE DATA Addr=253 Data=a3

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=235 wr_en_a=1 DATA=b5 addr_a=139

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=139 Data=b5

[SCB] PORT_A STORE DATA Addr=139 Data=b5

[SCB] PORT_B STORE DATA Addr=195 Data=a0

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=265 wr_en_a=1 DATA=4c addr_a=246

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=246 Data=4c

[SCB] PORT_A STORE DATA Addr=246 Data=4c

[SCB] PORT_B STORE DATA Addr=24 Data=67

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=295 wr_en_a=1 DATA=f6 addr_a=143

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=143 Data=f6

[SCB] PORT_A STORE DATA Addr=143 Data=f6

[SCB] PORT_B STORE DATA Addr=198 Data=6a

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=325 wr_en_a=1 DATA=12 addr_a=35

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=35 Data=12

[SCB] PORT_A STORE DATA Addr=35 Data=12

[SCB] PORT_B STORE DATA Addr=91 Data=c5

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=375 Addr=101 Data=98

[MON-RD] PORT_A_READ Addr=101 Data=98

[SCB] PORT_A [PASS] Addr=101 Data=98

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=415 Addr=237 Data=d5

[MON-RD] PORT_A_READ Addr=237 Data=d5

[SCB] PORT_A [PASS] Addr=237 Data=d5

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=455 Addr=14 Data=06

[MON-RD] PORT_A_READ Addr=14 Data=06

[SCB] PORT_A [PASS] Addr=14 Data=06

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=495 Addr=226 Data=fb

[MON-RD] PORT_A_READ Addr=226 Data=fb

[SCB] PORT_A [PASS] Addr=226 Data=fb

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=535 Addr=141 Data=2e

[MON-RD] PORT_A_READ Addr=141 Data=2e

[SCB] PORT_A [PASS] Addr=141 Data=2e

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=575 Addr=197 Data=f5

[MON-RD] PORT_A_READ Addr=197 Data=f5

[SCB] PORT_A [PASS] Addr=197 Data=f5

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=615 Addr=139 Data=b5

[MON-RD] PORT_A_READ Addr=139 Data=b5

[SCB] PORT_A [PASS] Addr=139 Data=b5

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=655 Addr=246 Data=4c

[MON-RD] PORT_A_READ Addr=246 Data=4c

[SCB] PORT_A [PASS] Addr=246 Data=4c

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=695 Addr=143 Data=f6

[MON-RD] PORT_A_READ Addr=143 Data=f6

[SCB] PORT_A [PASS] Addr=143 Data=f6

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=735 Addr=35 Data=12

[MON-RD] PORT_A_READ Addr=35 Data=12

[SCB] PORT_A [PASS] Addr=35 Data=12

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=755 wr_en_a=1 DATA=4e addr_a=11

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=11 Data=4e

[SCB] PORT_A STORE DATA Addr=11 Data=4e

[SCB] PORT_B STORE DATA Addr=134 Data=b7

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=805 Addr=11 Data=4e

[MON-RD] PORT_A_READ Addr=11 Data=4e

[SCB] PORT_A [PASS] Addr=11 Data=4e

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=825 wr_en_a=1 DATA=84 addr_a=209

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=209 Data=84

[SCB] PORT_A STORE DATA Addr=209 Data=84

[SCB] PORT_B STORE DATA Addr=81 Data=30

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=875 Addr=209 Data=84

[MON-RD] PORT_A_READ Addr=209 Data=84

[SCB] PORT_A [PASS] Addr=209 Data=84

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=895 wr_en_a=1 DATA=07 addr_a=238

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=238 Data=07

[SCB] PORT_A STORE DATA Addr=238 Data=07

[SCB] PORT_B STORE DATA Addr=255 Data=85

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=945 Addr=238 Data=07

[MON-RD] PORT_A_READ Addr=238 Data=07

[SCB] PORT_A [PASS] Addr=238 Data=07

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=965 wr_en_a=1 DATA=d3 addr_a=116

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=116 Data=d3

[SCB] PORT_A STORE DATA Addr=116 Data=d3

[SCB] PORT_B STORE DATA Addr=43 Data=21

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=1015 Addr=116 Data=d3

[MON-RD] PORT_A_READ Addr=116 Data=d3

[SCB] PORT_A [PASS] Addr=116 Data=d3

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=1035 wr_en_a=1 DATA=93 addr_a=200

[ASSERT PASS] Output stable when no read

[MON-WR] PORT_A_WRITE Addr=200 Data=93

[SCB] PORT_A STORE DATA Addr=200 Data=93

[SCB] PORT_B STORE DATA Addr=48 Data=44

[ASSERT PASS] Port A write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=1085 Addr=200 Data=93

[MON-RD] PORT_A_READ Addr=200 Data=93

[SCB] PORT_A [PASS] Addr=200 Data=93

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1115 wr_en_b=1 DATA=64 addr_b=0

[MON-WR] PORT_A_WRITE Addr=173 Data=0b

[SCB] PORT_A STORE DATA Addr=173 Data=0b

[SCB] PORT_B STORE DATA Addr=0 Data=64

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1145 wr_en_b=1 DATA=f9 addr_b=1

[MON-WR] PORT_A_WRITE Addr=136 Data=5d

[SCB] PORT_A STORE DATA Addr=136 Data=5d

[SCB] PORT_B STORE DATA Addr=1 Data=f9

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1175 wr_en_b=1 DATA=a8 addr_b=2

[MON-WR] PORT_A_WRITE Addr=79 Data=fa

[SCB] PORT_A STORE DATA Addr=79 Data=fa

[SCB] PORT_B STORE DATA Addr=2 Data=a8

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1205 wr_en_b=1 DATA=f1 addr_b=3

[MON-WR] PORT_A_WRITE Addr=184 Data=4a

[SCB] PORT_A STORE DATA Addr=184 Data=4a

[SCB] PORT_B STORE DATA Addr=3 Data=f1

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1235 wr_en_b=1 DATA=fc addr_b=4

[MON-WR] PORT_A_WRITE Addr=202 Data=ae

[SCB] PORT_A STORE DATA Addr=202 Data=ae

[SCB] PORT_B STORE DATA Addr=4 Data=fc

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1265 wr_en_b=1 DATA=26 addr_b=5

[MON-WR] PORT_A_WRITE Addr=161 Data=db

[SCB] PORT_A STORE DATA Addr=161 Data=db

[SCB] PORT_B STORE DATA Addr=5 Data=26

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1295 wr_en_b=1 DATA=28 addr_b=6

[MON-WR] PORT_A_WRITE Addr=26 Data=7c

[SCB] PORT_A STORE DATA Addr=26 Data=7c

[SCB] PORT_B STORE DATA Addr=6 Data=28

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1325 wr_en_b=1 DATA=e6 addr_b=7

[MON-WR] PORT_A_WRITE Addr=178 Data=24

[SCB] PORT_A STORE DATA Addr=178 Data=24

[SCB] PORT_B STORE DATA Addr=7 Data=e6

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1355 wr_en_b=1 DATA=90 addr_b=8

[MON-WR] PORT_A_WRITE Addr=140 Data=44

[SCB] PORT_A STORE DATA Addr=140 Data=44

[SCB] PORT_B STORE DATA Addr=8 Data=90

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1385 wr_en_b=1 DATA=47 addr_b=9

[MON-WR] PORT_A_WRITE Addr=140 Data=b9

[SCB] PORT_A STORE DATA Addr=140 Data=b9

[SCB] PORT_B STORE DATA Addr=9 Data=47

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[MON-RD] PORT_B_READ Addr=0 Data=64

[SCB] PORT_B [PASS] Addr=0 Data=64

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1435 Addr=0 Data=64

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=1 Data=f9

[SCB] PORT_B [PASS] Addr=1 Data=f9

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1475 Addr=1 Data=f9

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=2 Data=a8

[SCB] PORT_B [PASS] Addr=2 Data=a8

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1515 Addr=2 Data=a8

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=3 Data=f1

[SCB] PORT_B [PASS] Addr=3 Data=f1

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1555 Addr=3 Data=f1

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=4 Data=fc

[SCB] PORT_B [PASS] Addr=4 Data=fc

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1595 Addr=4 Data=fc

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=5 Data=26

[SCB] PORT_B [PASS] Addr=5 Data=26

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1635 Addr=5 Data=26

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=6 Data=28

[SCB] PORT_B [PASS] Addr=6 Data=28

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1675 Addr=6 Data=28

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=7 Data=e6

[SCB] PORT_B [PASS] Addr=7 Data=e6

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1715 Addr=7 Data=e6

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=8 Data=90

[SCB] PORT_B [PASS] Addr=8 Data=90

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1755 Addr=8 Data=90

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=9 Data=47

[SCB] PORT_B [PASS] Addr=9 Data=47

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1795 Addr=9 Data=47

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1815 wr_en_b=1 DATA=a0 addr_b=0

[MON-WR] PORT_A_WRITE Addr=53 Data=f5

[SCB] PORT_A STORE DATA Addr=53 Data=f5

[SCB] PORT_B STORE DATA Addr=0 Data=a0

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1845 wr_en_b=1 DATA=a2 addr_b=1

[MON-WR] PORT_A_WRITE Addr=202 Data=b4

[SCB] PORT_A STORE DATA Addr=202 Data=b4

[SCB] PORT_B STORE DATA Addr=1 Data=a2

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1875 wr_en_b=1 DATA=a4 addr_b=2

[MON-WR] PORT_A_WRITE Addr=14 Data=e9

[SCB] PORT_A STORE DATA Addr=14 Data=e9

[SCB] PORT_B STORE DATA Addr=2 Data=a4

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1905 wr_en_b=1 DATA=a6 addr_b=3

[MON-WR] PORT_A_WRITE Addr=211 Data=e2

[SCB] PORT_A STORE DATA Addr=211 Data=e2

[SCB] PORT_B STORE DATA Addr=3 Data=a6

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=1935 wr_en_b=1 DATA=a8 addr_b=4

[MON-WR] PORT_A_WRITE Addr=102 Data=d5

[SCB] PORT_A STORE DATA Addr=102 Data=d5

[SCB] PORT_B STORE DATA Addr=4 Data=a8

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[MON-RD] PORT_B_READ Addr=0 Data=a0

[SCB] PORT_B [PASS] Addr=0 Data=a0

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=1985 Addr=0 Data=a0

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=1 Data=a2

[SCB] PORT_B [PASS] Addr=1 Data=a2

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2025 Addr=1 Data=a2

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=2 Data=a4

[SCB] PORT_B [PASS] Addr=2 Data=a4

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2065 Addr=2 Data=a4

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=3 Data=a6

[SCB] PORT_B [PASS] Addr=3 Data=a6

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2105 Addr=3 Data=a6

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=4 Data=a8

[SCB] PORT_B [PASS] Addr=4 Data=a8

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2145 Addr=4 Data=a8

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2165 wr_en_b=1 DATA=b0 addr_b=0

[MON-WR] PORT_A_WRITE Addr=83 Data=1f

[SCB] PORT_A STORE DATA Addr=83 Data=1f

[SCB] PORT_B STORE DATA Addr=0 Data=b0

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2195 wr_en_b=1 DATA=b2 addr_b=1

[MON-WR] PORT_A_WRITE Addr=79 Data=d4

[SCB] PORT_A STORE DATA Addr=79 Data=d4

[SCB] PORT_B STORE DATA Addr=1 Data=b2

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2225 wr_en_b=1 DATA=b4 addr_b=2

[MON-WR] PORT_A_WRITE Addr=246 Data=9e

[SCB] PORT_A STORE DATA Addr=246 Data=9e

[SCB] PORT_B STORE DATA Addr=2 Data=b4

[ASSERT PASS] Port B write detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[MON-RD] PORT_B_READ Addr=0 Data=b0

[SCB] PORT_B [PASS] Addr=0 Data=b0

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2275 Addr=0 Data=b0

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=1 Data=b2

[SCB] PORT_B [PASS] Addr=1 Data=b2

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2315 Addr=1 Data=b2

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=2 Data=b4

[SCB] PORT_B [PASS] Addr=2 Data=b4

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2355 Addr=2 Data=b4

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=3 Data=a6

[SCB] PORT_B [PASS] Addr=3 Data=a6

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2395 Addr=3 Data=a6

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=4 Data=a8

[SCB] PORT_B [PASS] Addr=4 Data=a8

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2435 Addr=4 Data=a8

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=5 Data=26

[SCB] PORT_B [PASS] Addr=5 Data=26

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2475 Addr=5 Data=26

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[MON-RD] PORT_B_READ Addr=6 Data=28

[SCB] PORT_B [PASS] Addr=6 Data=28

[ASSERT PASS] Port B read valid

[DRV] READ DATA_PORT_B OK : Time=2515 Addr=6 Data=28

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2525 wr_en_a=1 DATA=a4 addr_a=0

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2535 wr_en_b=1 DATA=bc addr_b=10

[MON-WR] PORT_A_WRITE Addr=0 Data=a4

[SCB] PORT_A STORE DATA Addr=0 Data=a4

[SCB] PORT_B STORE DATA Addr=10 Data=bc

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT PASS] Both ports writing different addresses

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2555 wr_en_a=1 DATA=85 addr_a=1

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2565 wr_en_b=1 DATA=7d addr_b=11

[MON-WR] PORT_A_WRITE Addr=1 Data=85

[SCB] PORT_A STORE DATA Addr=1 Data=85

[SCB] PORT_B STORE DATA Addr=11 Data=7d

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT PASS] Both ports writing different addresses

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2585 wr_en_a=1 DATA=a9 addr_a=2

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2595 wr_en_b=1 DATA=93 addr_b=12

[MON-WR] PORT_A_WRITE Addr=2 Data=a9

[SCB] PORT_A STORE DATA Addr=2 Data=a9

[SCB] PORT_B STORE DATA Addr=12 Data=93

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT PASS] Both ports writing different addresses

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2615 wr_en_a=1 DATA=07 addr_a=3

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2625 wr_en_b=1 DATA=6b addr_b=13

[MON-WR] PORT_A_WRITE Addr=3 Data=07

[SCB] PORT_A STORE DATA Addr=3 Data=07

[SCB] PORT_B STORE DATA Addr=13 Data=6b

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT PASS] Both ports writing different addresses

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2645 wr_en_a=1 DATA=70 addr_a=4

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2655 wr_en_b=1 DATA=eb addr_b=14

[MON-WR] PORT_A_WRITE Addr=4 Data=70

[SCB] PORT_A STORE DATA Addr=4 Data=70

[SCB] PORT_B STORE DATA Addr=14 Data=eb

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT PASS] Both ports writing different addresses

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[ASSERT PASS] Read after write on Port B

[DRV] READ DATA_PORT_A OK : Time=2695 Addr=0 Data=a4

[MON-RD] PORT_A_READ Addr=0 Data=a4

[SCB] PORT_A [PASS] Addr=0 Data=a4

[ASSERT PASS] Port A read valid

[ASSERT PASS] Port B read valid

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_B OK : Time=2715 Addr=10 Data=bc

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=2745 Addr=1 Data=85

[MON-RD] PORT_A_READ Addr=1 Data=85

[SCB] PORT_A [PASS] Addr=1 Data=85

[ASSERT PASS] Port A read valid

[ASSERT PASS] Port B read valid

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_B OK : Time=2765 Addr=11 Data=7d

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=2795 Addr=2 Data=a9

[MON-RD] PORT_A_READ Addr=2 Data=a9

[SCB] PORT_A [PASS] Addr=2 Data=a9

[ASSERT PASS] Port A read valid

[ASSERT PASS] Port B read valid

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_B OK : Time=2815 Addr=12 Data=93

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=2845 Addr=3 Data=07

[MON-RD] PORT_A_READ Addr=3 Data=07

[SCB] PORT_A [PASS] Addr=3 Data=07

[ASSERT PASS] Port A read valid

[ASSERT PASS] Port B read valid

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_B OK : Time=2865 Addr=13 Data=6b

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=2895 Addr=4 Data=70

[MON-RD] PORT_A_READ Addr=4 Data=70

[SCB] PORT_A [PASS] Addr=4 Data=70

[ASSERT PASS] Port A read valid

[ASSERT PASS] Port B read valid

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_B OK : Time=2915 Addr=14 Data=eb

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2925 wr_en_a=1 DATA=46 addr_a=0

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2935 wr_en_b=1 DATA=50 addr_b=0

[MON-WR] PORT_A_WRITE Addr=0 Data=46

[SCB] PORT_A STORE DATA Addr=0 Data=46

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT INFO] Write conflict detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2955 wr_en_a=1 DATA=47 addr_a=1

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2965 wr_en_b=1 DATA=51 addr_b=1

[MON-WR] PORT_A_WRITE Addr=1 Data=47

[SCB] PORT_A STORE DATA Addr=1 Data=47

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT INFO] Write conflict detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=2985 wr_en_a=1 DATA=48 addr_a=2

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=2995 wr_en_b=1 DATA=52 addr_b=2

[MON-WR] PORT_A_WRITE Addr=2 Data=48

[SCB] PORT_A STORE DATA Addr=2 Data=48

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT INFO] Write conflict detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=3015 wr_en_a=1 DATA=49 addr_a=3

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=3025 wr_en_b=1 DATA=53 addr_b=3

[MON-WR] PORT_A_WRITE Addr=3 Data=49

[SCB] PORT_A STORE DATA Addr=3 Data=49

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT INFO] Write conflict detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_A : Time=3045 wr_en_a=1 DATA=4a addr_a=4

[ASSERT PASS] Output stable when no read

[DRV] WRITE DATA_PORT_B : Time=3055 wr_en_b=1 DATA=54 addr_b=4

[MON-WR] PORT_A_WRITE Addr=4 Data=4a

[SCB] PORT_A STORE DATA Addr=4 Data=4a

[ASSERT PASS] Port A write detected

[ASSERT PASS] Port B write detected

[ASSERT INFO] Write conflict detected

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[ASSERT PASS] Read after write on Port A

[DRV] READ DATA_PORT_A OK : Time=3095 Addr=0 Data=46

[MON-RD] PORT_A_READ Addr=0 Data=46

[SCB] PORT_A [PASS] Addr=0 Data=46

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=3135 Addr=1 Data=47

[MON-RD] PORT_A_READ Addr=1 Data=47

[SCB] PORT_A [PASS] Addr=1 Data=47

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=3175 Addr=2 Data=48

[MON-RD] PORT_A_READ Addr=2 Data=48

[SCB] PORT_A [PASS] Addr=2 Data=48

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=3215 Addr=3 Data=49

[MON-RD] PORT_A_READ Addr=3 Data=49

[SCB] PORT_A [PASS] Addr=3 Data=49

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[DRV] READ DATA_PORT_A OK : Time=3255 Addr=4 Data=4a

[MON-RD] PORT_A_READ Addr=4 Data=4a

[SCB] PORT_A [PASS] Addr=4 Data=4a

[ASSERT PASS] Port A read valid

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

[ASSERT PASS] Output stable when no read

V C S S i m u l a t i o n R e p o r t

Time: 4045

Assertions of RAM

```
//
module ram_assertions (
        input logic clk,
      input logic rst,

      input logic wr_en_a,
      input logic rd_en_a,
      input logic wr_en_b,
      input logic rd_en_b,

      input logic [7:0] addr_a,
      input logic [7:0] addr_b,

      input logic [7:0] data_in_a,
      input logic [7:0] data_in_b,

      input logic [7:0] data_out_a,
      input logic [7:0] data_out_b
);

property reset_check;
  @(posedge clk)
  rst |-> (data_out_a == 0 && data_out_b == 0);
endproperty
```

```
assert property(reset_check)
$display("[ASSERT PASS] Reset clears outputs");

property portA_write;
  @(posedge clk)
  wr_en_a |-> 1;
endproperty

assert property(portA_write)
$display("[ASSERT PASS] Port A write detected");

property portB_write;
  @(posedge clk)
  wr_en_b |-> 1;
endproperty

assert property(portB_write)
$display("[ASSERT PASS] Port B write detected");

property portA_read;
  @(posedge clk)
  rd_en_a |-> ##1 (data_out_a !== 'x);
endproperty

assert property(portA_read)
$display("[ASSERT PASS] Port A read valid");
```

```
property portB_read;
  @(posedge clk)
  rd_en_b |-> ##1 (data_out_b !== 'x);
endproperty

assert property(portB_read)
$display("[ASSERT PASS] Port B read valid");

property simultaneous_write_diff_addr;
  @(posedge clk)
  (wr_en_a && wr_en_b && addr_a != addr_b) |-> 1;
endproperty

assert property(simultaneous_write_diff_addr)
$display("[ASSERT PASS] Both ports writing different addresses");

property write_conflict;
  @(posedge clk)
  (wr_en_a && wr_en_b && addr_a == addr_b) |-> 1;
endproperty

assert property(write_conflict)
$display("[ASSERT INFO] Write conflict detected");

property read_after_write_A;
   @(posedge clk)
```

```
   wr_en_a |-> ##[1:$] rd_en_a;
endproperty

assert property(read_after_write_A)
$display("[ASSERT PASS] Read after write on Port A");

property read_after_write_B;
   @(posedge clk)
   wr_en_b |-> ##[1:$] rd_en_b;
endproperty

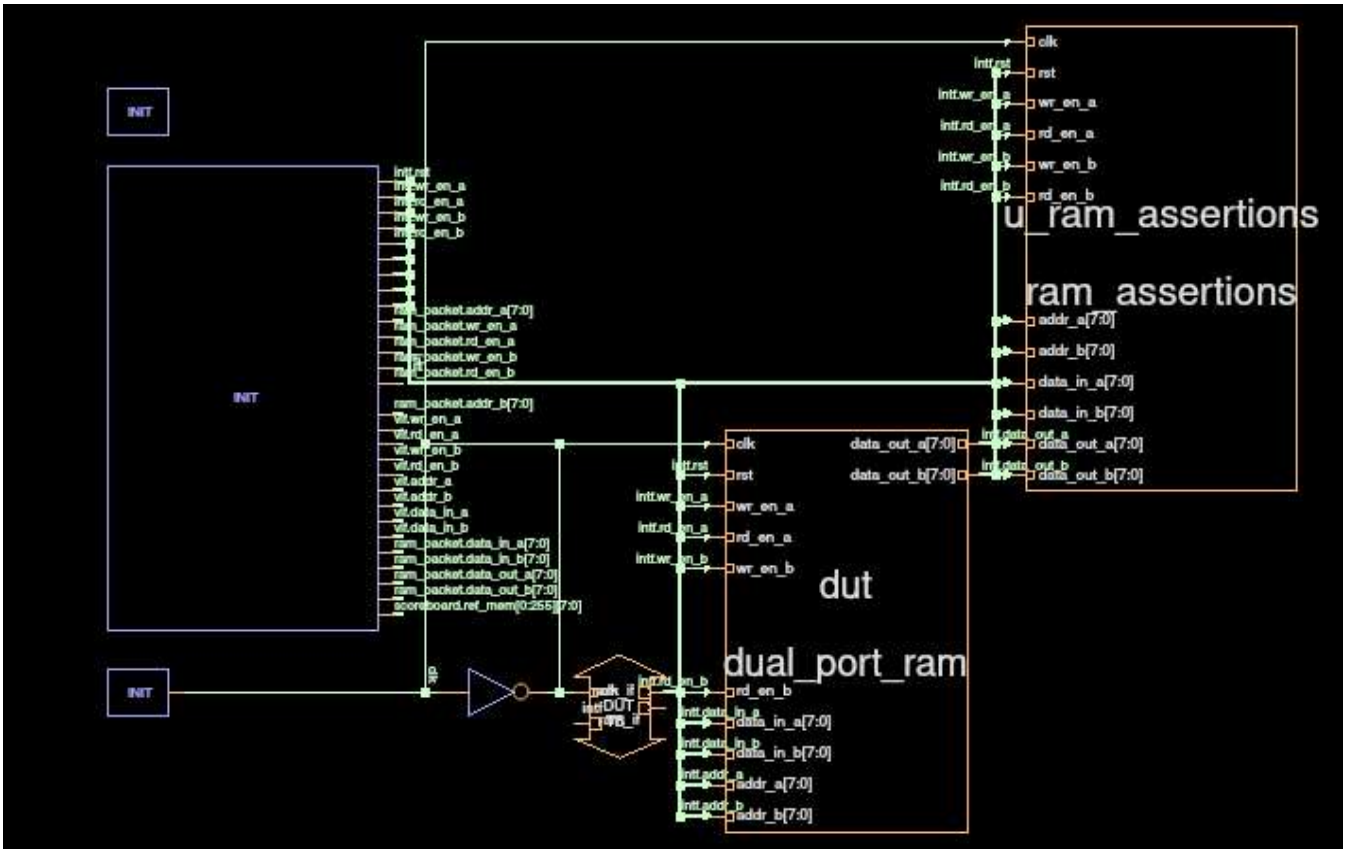
assert property(read_after_write_B)
$display("[ASSERT PASS] Read after write on Port B");

property output_stable;
  @(posedge clk)
  (!rd_en_a && !rd_en_b) |=> ($stable(data_out_a) && $stable(data_out_b));
endproperty

assert property(output_stable)
$display("[ASSERT PASS] Output stable when no read");

endmodule
```

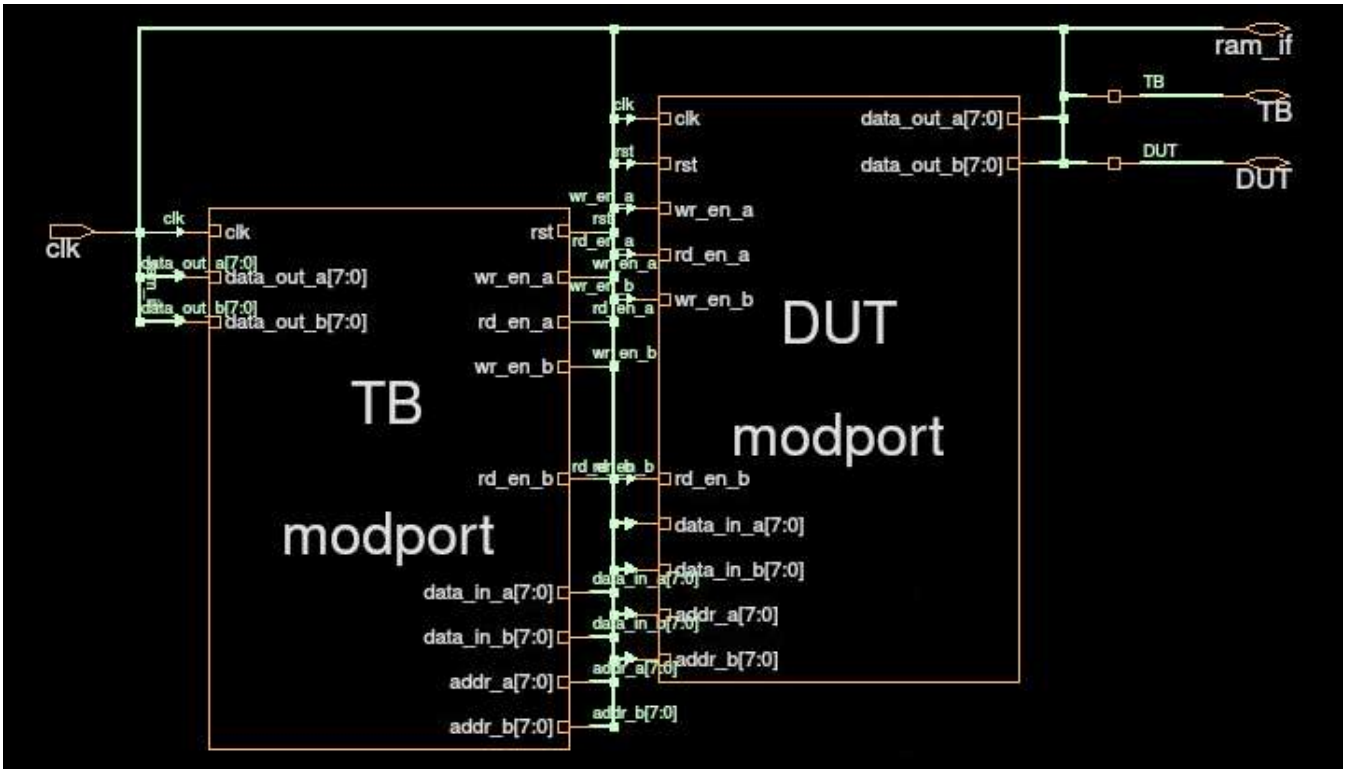
Schematic Design



(Internal Schematic View of synthesizable circuits of Dual-Port RAM - by VCS and Verdi tool)



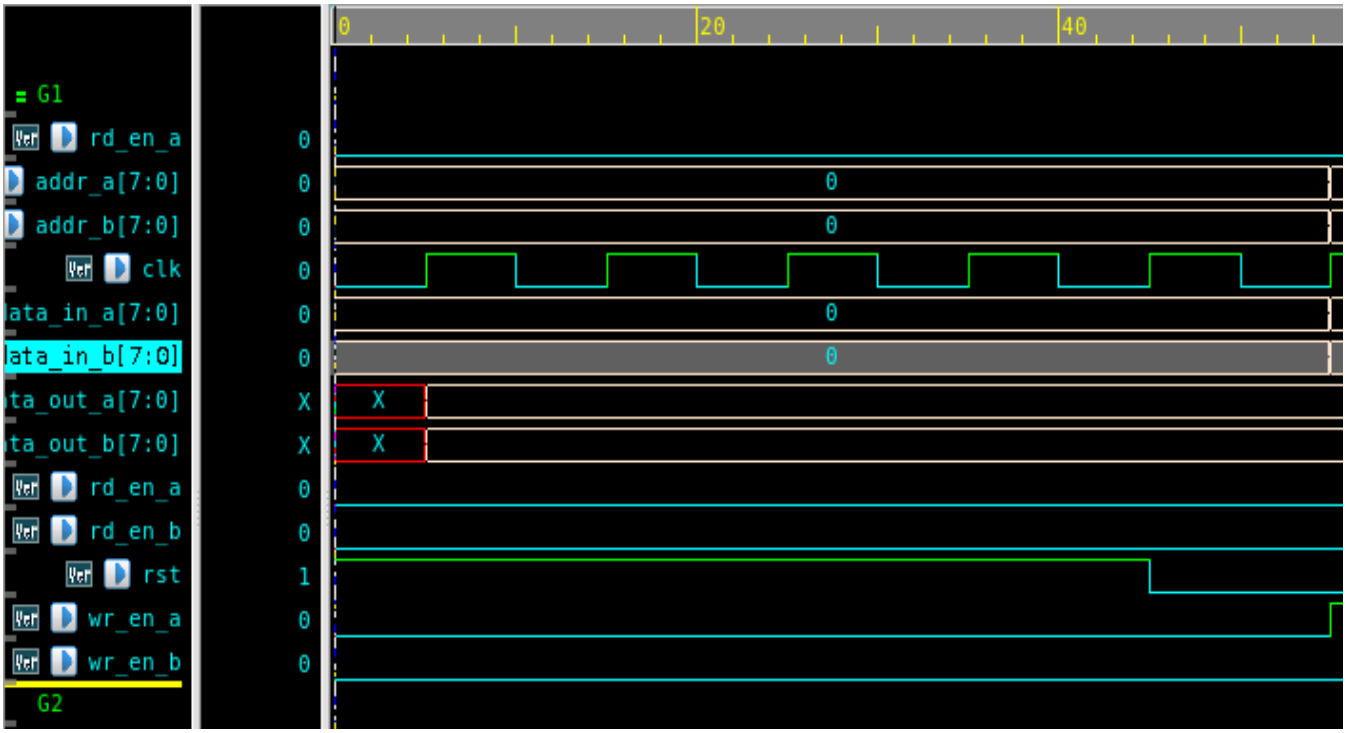
(256 ×8-bit Dual-Port RAM blocks)



(Internal Schematic View of DUT & TB circuits of Dual-Port RAM - by VCS and Verdi tool)

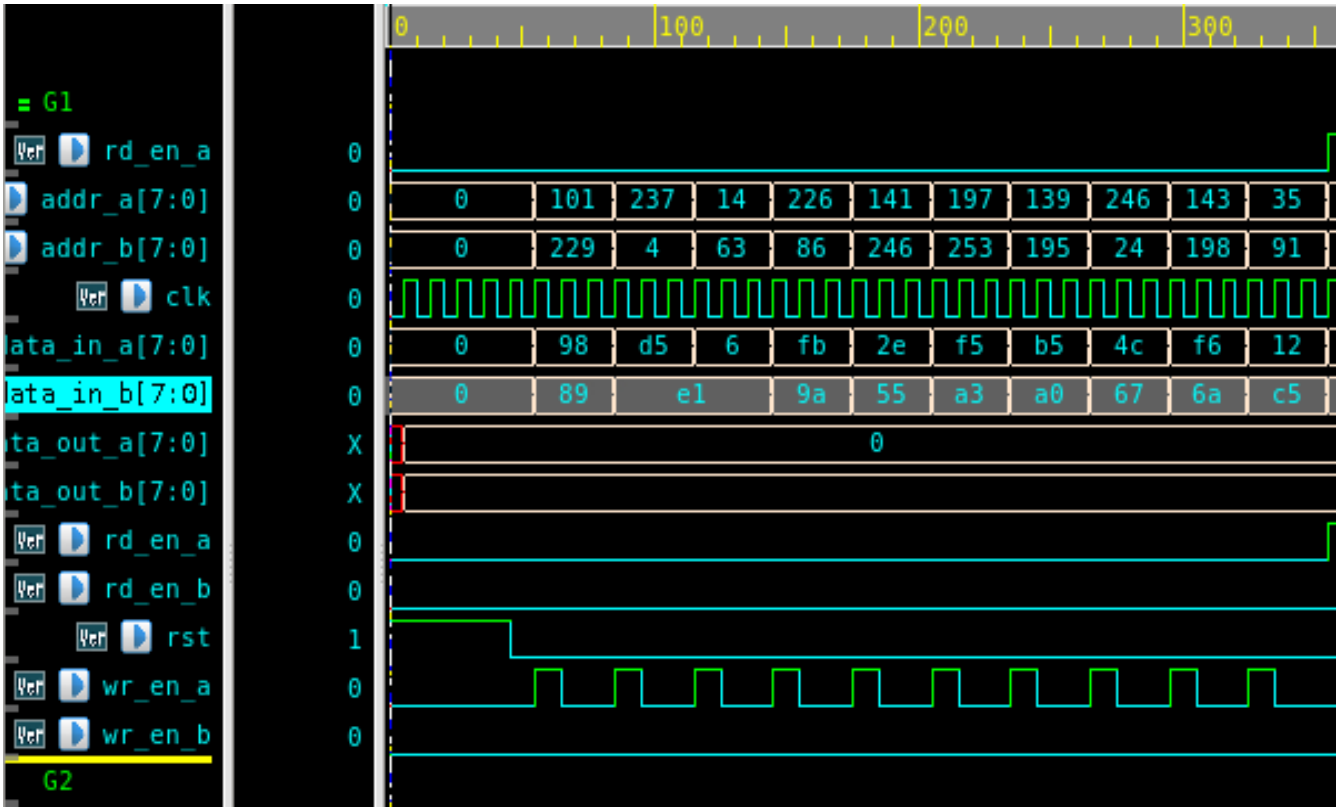
Waveform-

Test case1: Check the reset Condition



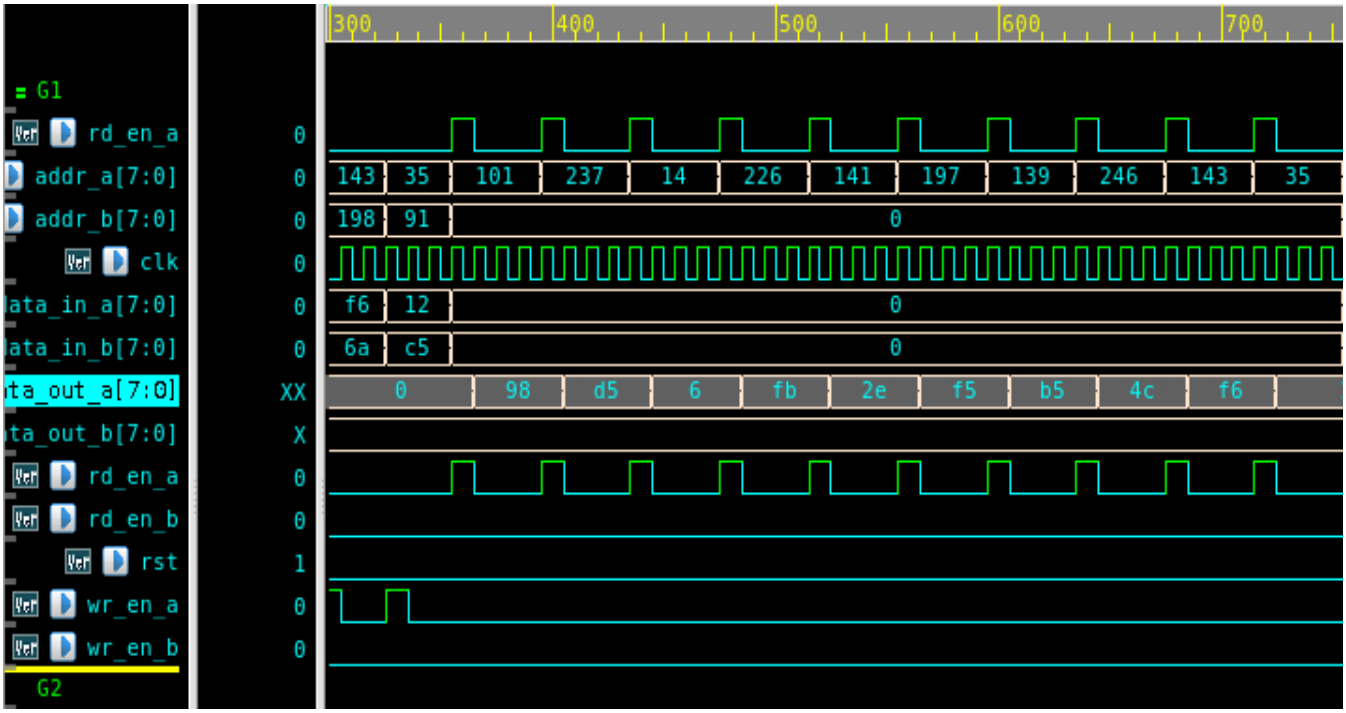
(Test status: checked pass)

Test case2: WRITE through PORT A



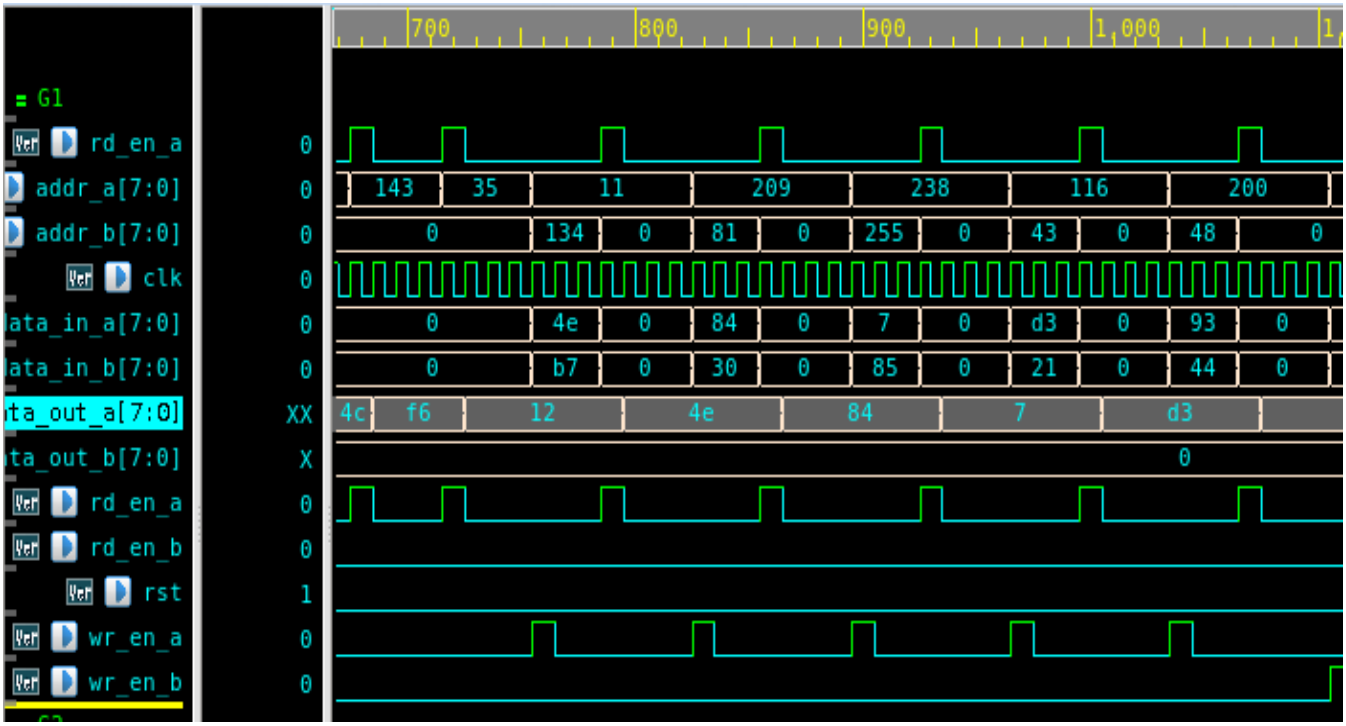
(Test status: checked pass)

Test case3: READ through PORT A



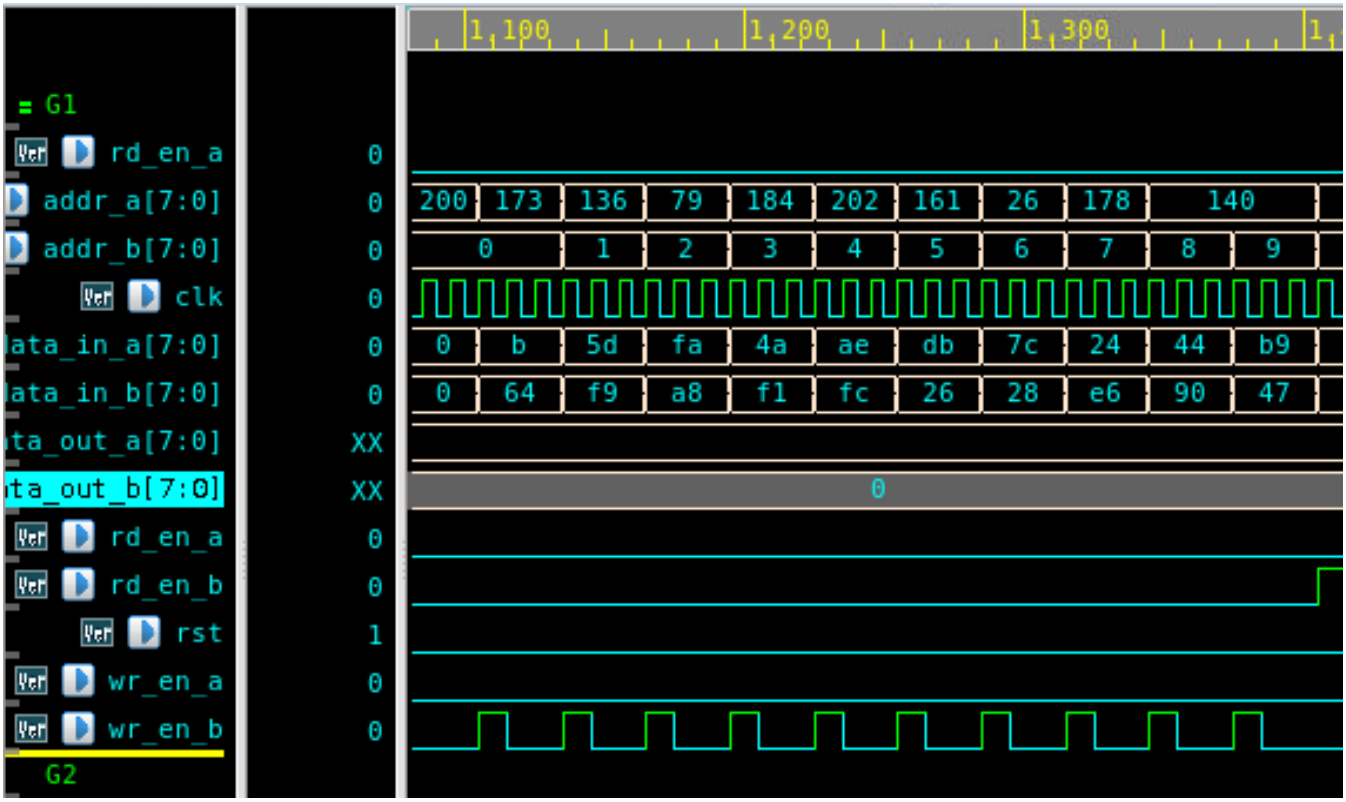
(Test status: checked pass)

Test case3: WRITE-READ through PORT A



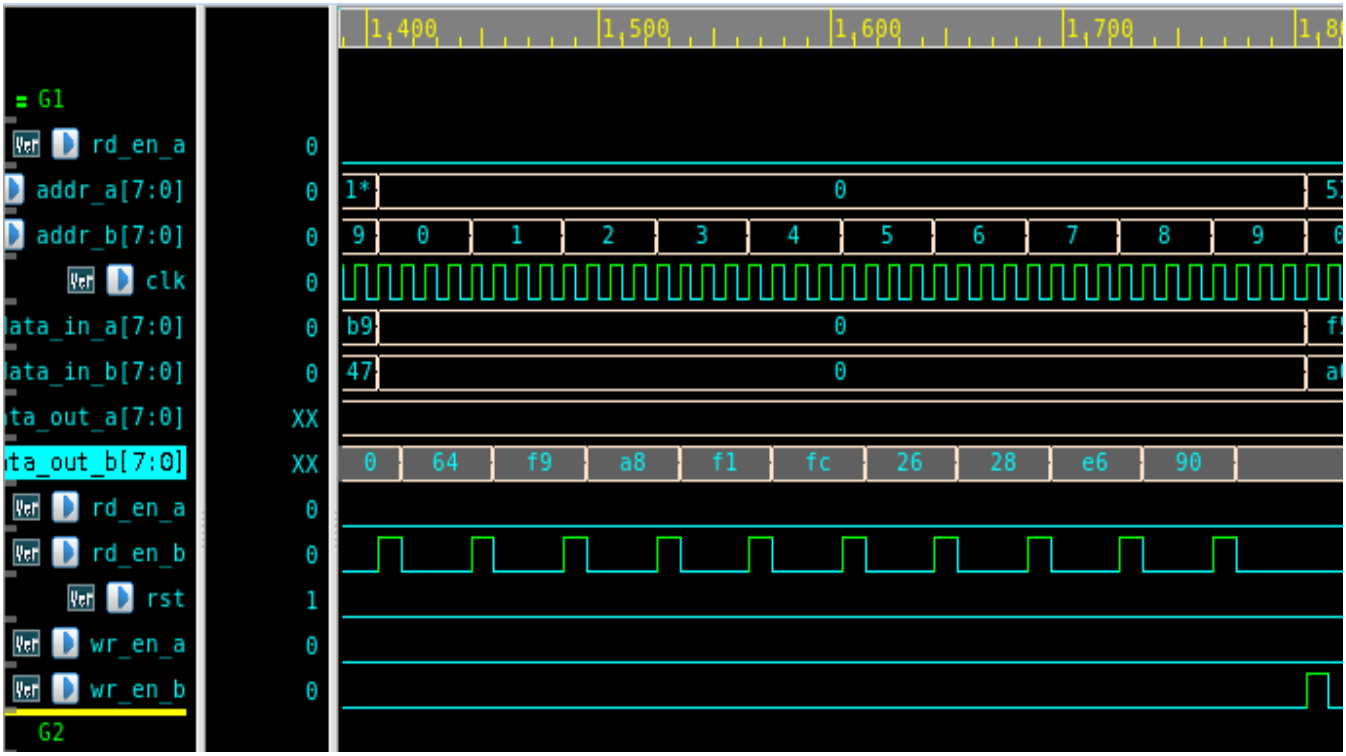
(Test status: checked pass)

Test case4: WRITE through PORT B



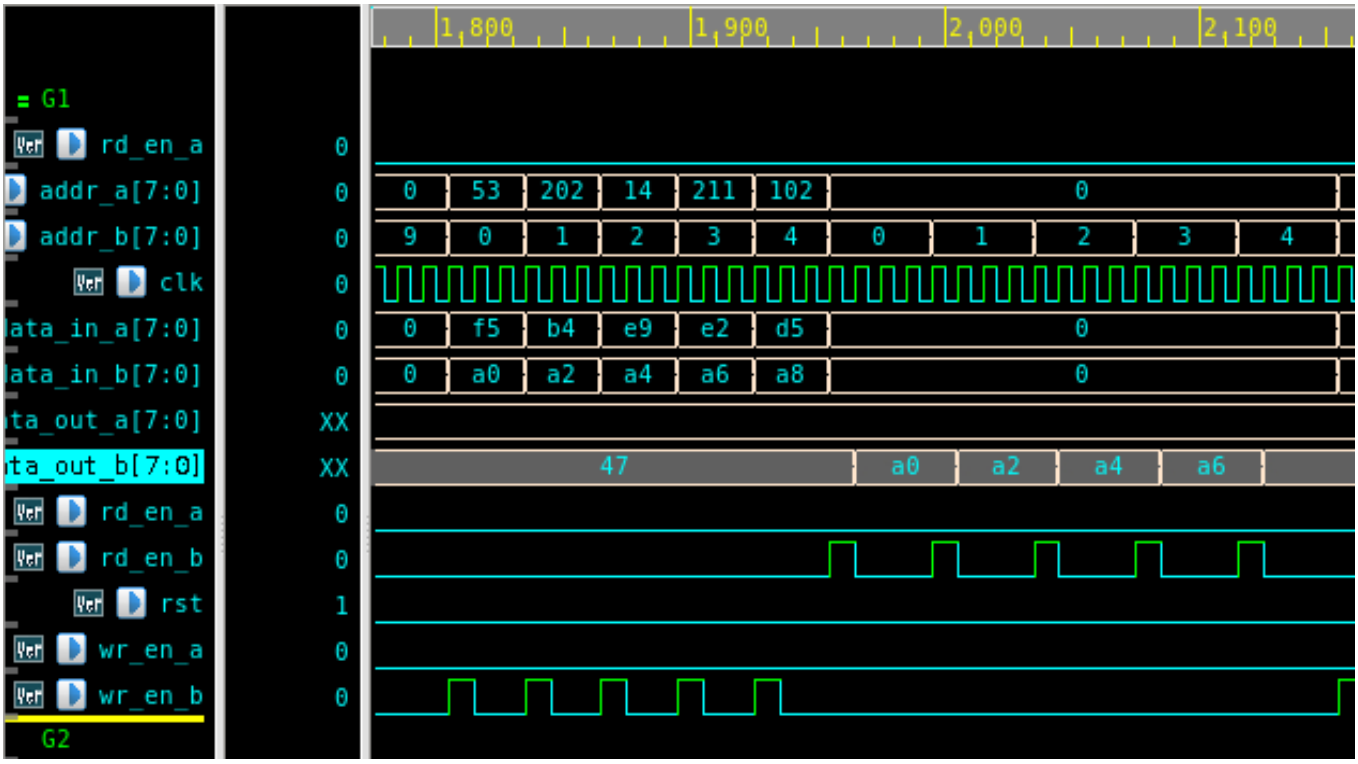
(Test status: checked pass)

Test case5: READ through PORT B



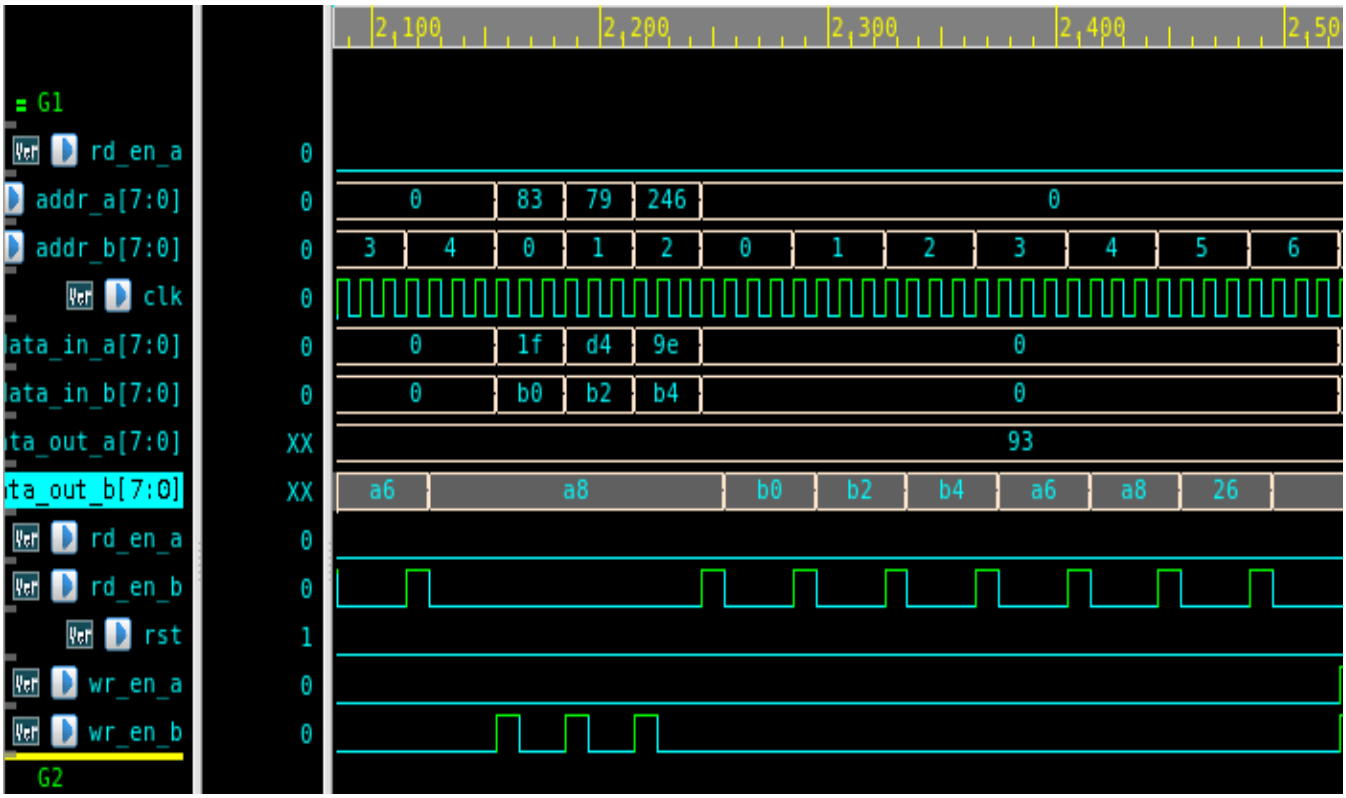
(Test status: checked pass)

Test case6: WRITE- READ through PORT B



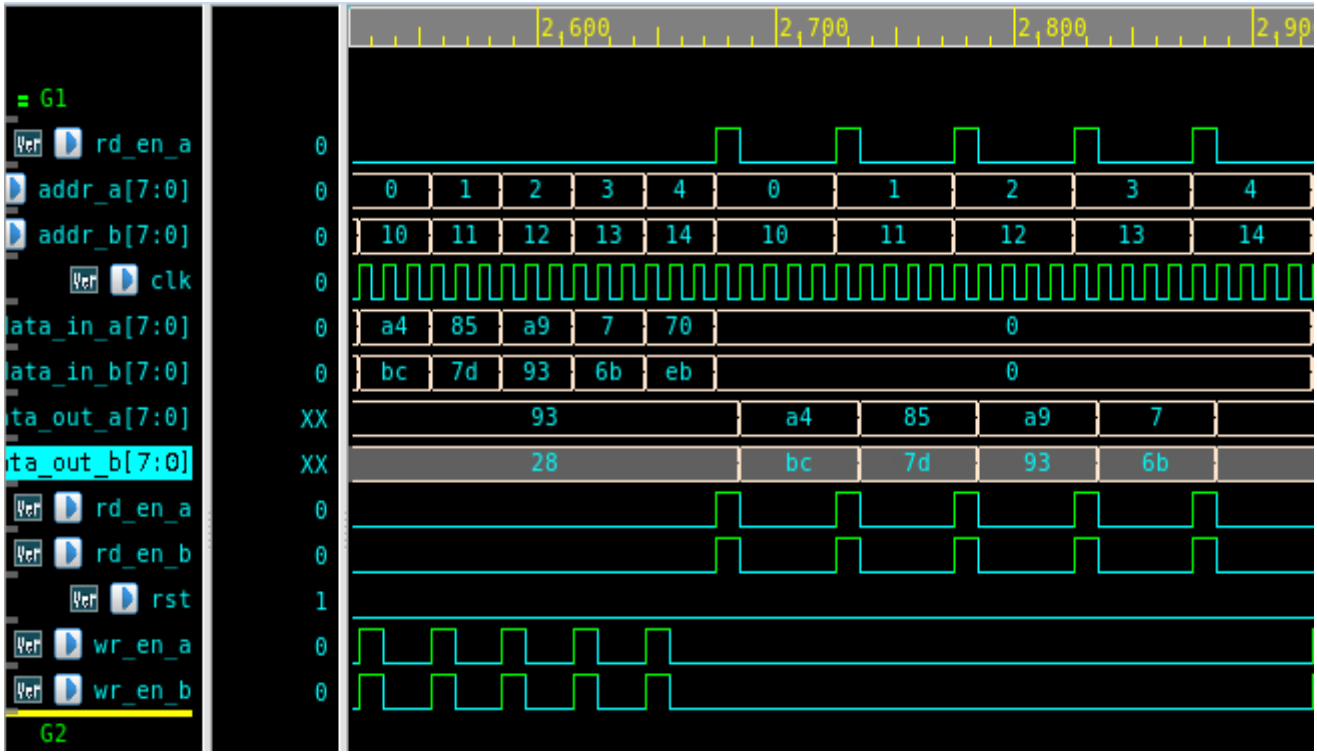
(Test status: checked pass)

Test case7: 3 OVER-WRITE 7 READ DATA PORT B



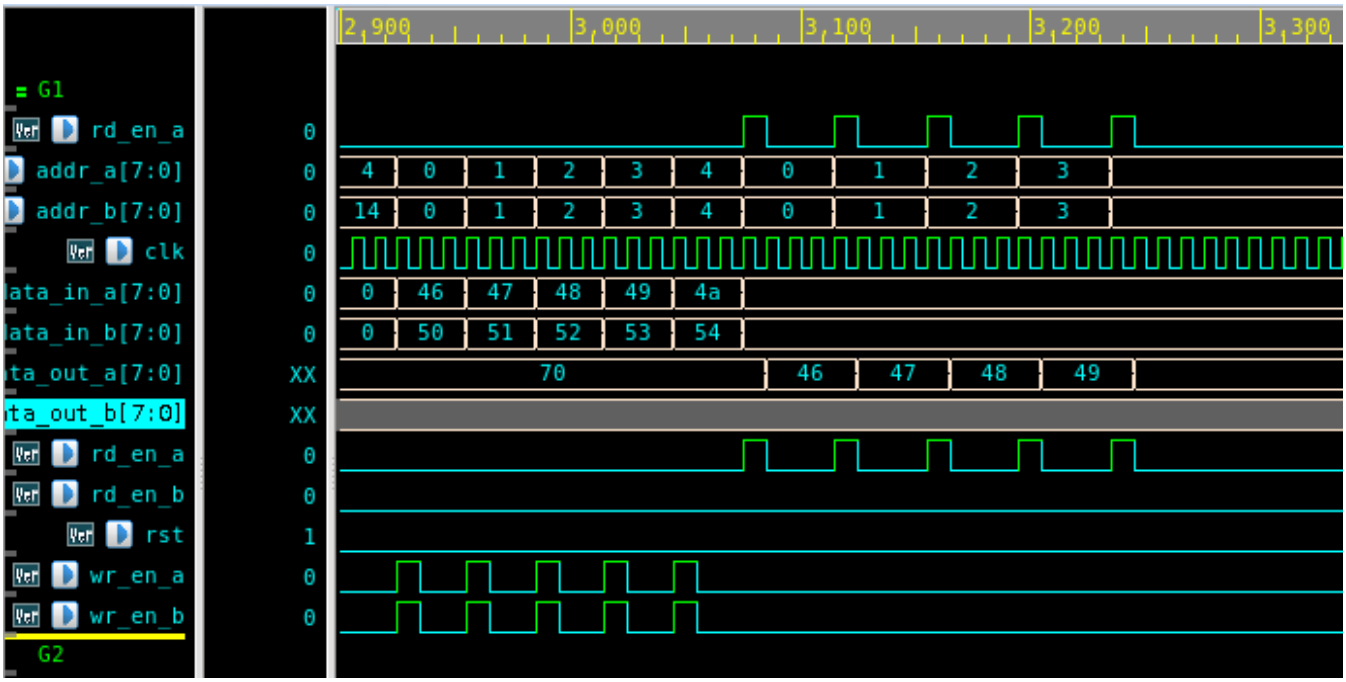
(Test status: checked pass)

Test case8: WRITE-READ through PORT A & PORT B



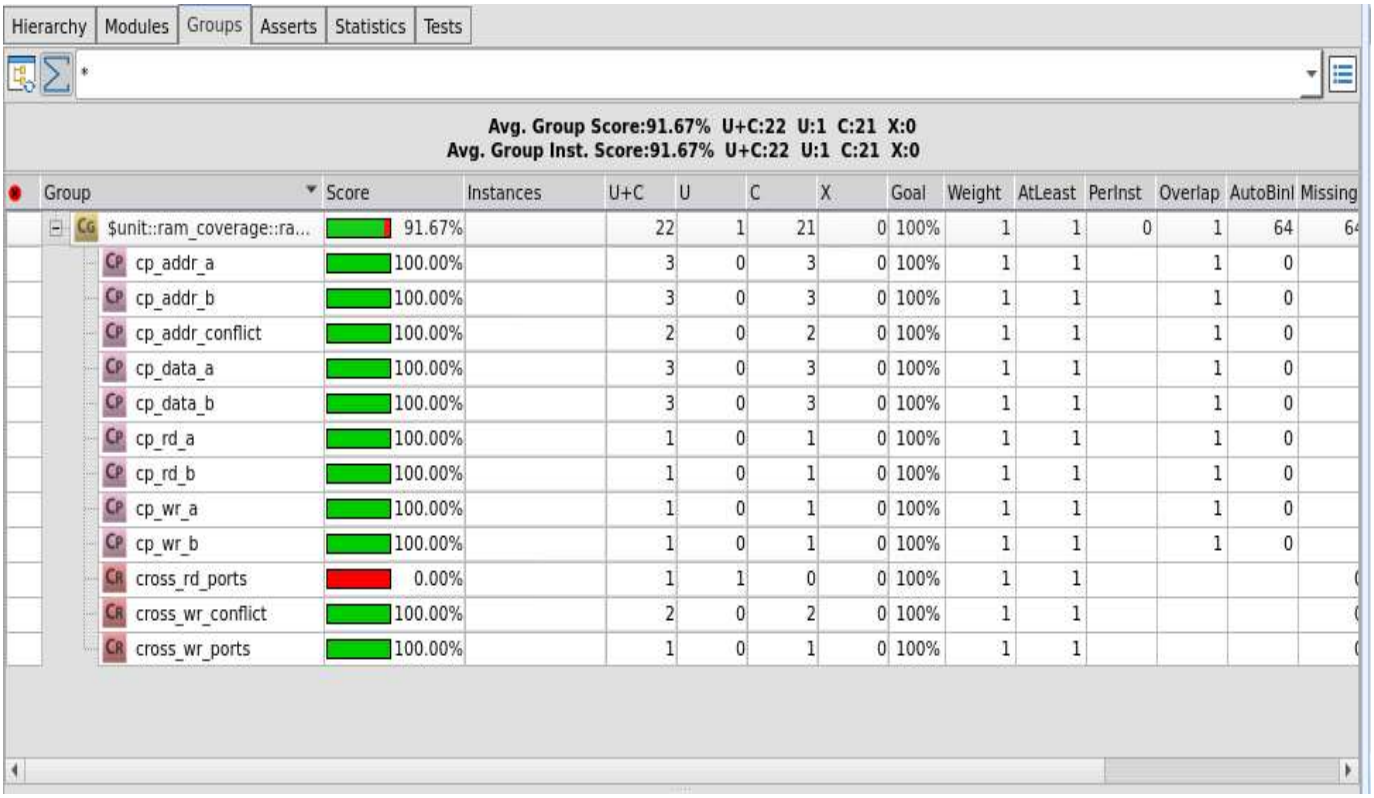
(Test status: checked pass)

Test case9: WRITE-READ through PORT A & PORT B at SAME address (Priority PORT A)



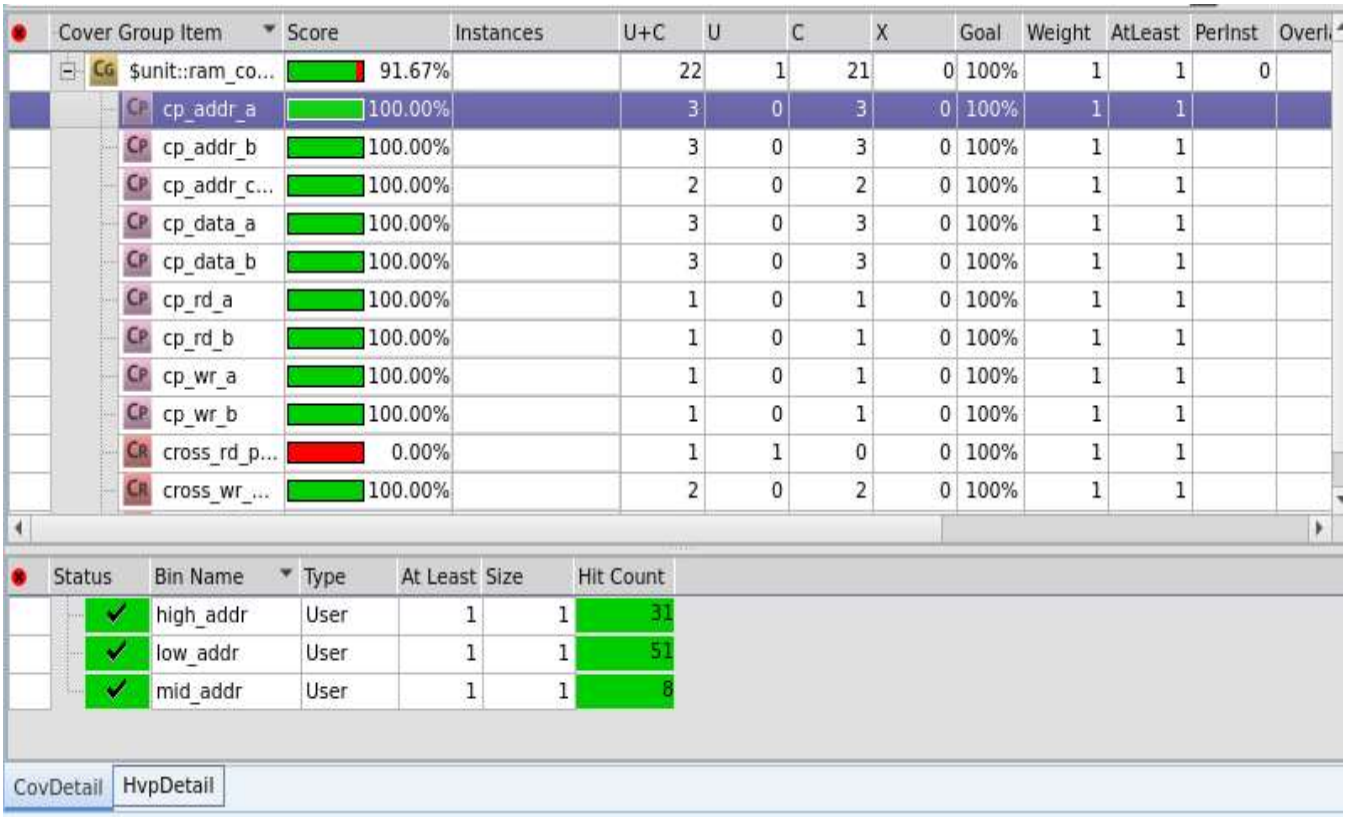
(Test status: checked pass)

Coverage Report:-



Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBinI	Missing
$unit::ram_coverage::ra...	91.67%		22	1	21	0	100%	1	1	0	1	64	64
cp_addr_a	100.00%		3	0	3	0	100%	1	1		1	0	
cp_addr_b	100.00%		3	0	3	0	100%	1	1		1	0	
cp_addr_conflict	100.00%		2	0	2	0	100%	1	1		1	0	
cp_data_a	100.00%		3	0	3	0	100%	1	1		1	0	
cp_data_b	100.00%		3	0	3	0	100%	1	1		1	0	
cp_rd_a	100.00%		1	0	1	0	100%	1	1		1	0	
cp_rd_b	100.00%		1	0	1	0	100%	1	1		1	0	
cp_wr_a	100.00%		1	0	1	0	100%	1	1		1	0	
cp_wr_b	100.00%		1	0	1	0	100%	1	1		1	0	
cross_rd_ports	0.00%		1	1	0	0	100%	1	1				0
cross_wr_conflict	100.00%		2	0	2	0	100%	1	1				0
cross_wr_ports	100.00%		1	0	1	0	100%	1	1				0

(Test status: coverage Score: 91.67%)



Cover Group Item	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overl
$unit::ram_co...	91.67%		22	1	21	0	100%	1	1	0	
cp_addr_a	100.00%		3	0	3	0	100%	1	1		
cp_addr_b	100.00%		3	0	3	0	100%	1	1		
cp_addr_c...	100.00%		2	0	2	0	100%	1	1		
cp_data_a	100.00%		3	0	3	0	100%	1	1		
cp_data_b	100.00%		3	0	3	0	100%	1	1		
cp_rd_a	100.00%		1	0	1	0	100%	1	1		
cp_rd_b	100.00%		1	0	1	0	100%	1	1		
cp_wr_a	100.00%		1	0	1	0	100%	1	1		
cp_wr_b	100.00%		1	0	1	0	100%	1	1		
cross_rd_p...	0.00%		1	1	0	0	100%	1	1		
cross_wr_...	100.00%		2	0	2	0	100%	1	1		

Status	Bin Name	Type	At Least	Size	Hit Count
✓	high_addr	User	1	1	31
✓	low_addr	User	1	1	51
✓	mid_addr	User	1	1	8

CovDetail HvpDetail

<u>Finally</u>, our design is functionally correct and passes all simulation cases: In conclusion, the Dual-Port RAM design has been thoroughly verified for all corner cases, including read and write operations, address reuse, overwrite scenarios, and data correctness. Dual-Port RAM plays a vital role in modern digital systems by allowing parallel memory access, improving performance and enabling efficient data sharing between multiple processing units and save power efficiently and fast processing. **Also,** I got the coverage score is around 91.67% which is the good for the design verification for Dual-Port RAM.